

Sprint 5 – Alien March with Observers

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies: Yes No

Name:

Date:

Submission Details

Final **Changelist** number:

Verified build: Yes No

Required Configurations:

YouTubeLink:

Discussion (What did you learn):

Verify Builds

- Follow the Piazza procedure on submission
 - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
 - No – Generated files
 - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
 - Anything that is generated by the compiler should not be included
 - No – Generated directories
 - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
 - *.sln, *.csproj, *.cs,
 - App.config, AssemblyInfo.cs, CleanMe.bat
 - Resources Directory:
 - *.tga, *.dll, *.wav, *.gls, *.azul

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perforce several times
 - Design Patterns – no minimum
 - Sprints – minimum of 5 real submissions (generally 10+)
 - As soon as you get something working, submit to perforce
 - Have reasonable check-in comments
 - Points will be deducted if minimum is not reached

Submission Report

- Fill out the submission Report
 - No report, no grade

Code and project needs to compile and run

- Make sure that your program compiles and runs
 - Warning level 4
 - NO Warnings or ERRORS
 - Your code should be squeaky clean.
 - Code needs to work “as-is”.
 - No modifications to files or deleting files necessary to compile or run.
 - All your code must compile from perforce with no modifications.
 - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason...
 - It will not be graded and you get a 0

No Containers

- Containers (No automatic containers or arrays)
- Template or generic parameters
- No arrays
 - You need to do this the old fashion way - **YOU EARNED IT**

Leave Project Settings

- Do NOT change the project or warning level
 - Any changing of level or suppression of warnings is an integrity issue

Simple C#

- No .Net
- We are using the basics
 - Types:
 - Class, Structs, intrinsic types (int, float, bool, etc...)
 - NO arrays allowed!
 - Basics language features
 - Inheritance, methods, abstract, virtual, etc...

No Debug code or files disabled

- Make sure the program has only active code
 - If you added debug code or commented out code,
 - please return to code to active state or remove it

Adding files to this project

- Make sure you add the files in the appropriate sub-directories
- Make sure any new files are successfully integrated into the project
- Make sure your new files are submitted to Perforce

Due Dates

- See Piazza for due date and time
- Submit program perforce in your student directory assignment supplied.
- Fill out your this **Submission Report** and commit to perforce
 - **ONLY** use Adobe Reader to fill out form, all others will be rejected.
 - Fill out the form and discussion for full credit.

Goals

- Observer working with the visitor pattern
- Font system with Flyweight Glyphs
- Sound system

Assignments

Grading:

High Level summary: https://youtu.be/xn1Ouc_CIGs

Show and tell: <https://youtu.be/MEIKrgNcvsQ>

- **Visitor working**
 - Detect collisions with a Tree vs Tree hierarchy composite pattern
 - Visitor help filter the correct reaction between two nodes on the tree
- **Observers working**
 - After detection of a collision
 - Notify all the listeners / observers to change the direction of the grid
- **Flyweight working**
 - Load up the Glyph manager as a flyweight
 - Use Glyph to render FontSprints
- **Snd System working**
 - Understand IrrKlang audio library
 - Play and adjust sound for the marching aliens

Required Features:

Collision system

- Need to have two Trees for Collision detection between the **AlienGrid** and the **WallGroup**
- Each tree needs to be constructed using **Composite Pattern**
- The collision system uses the **Visitor pattern** to filter the collision correctly
 - AlienGrid vs WallGroup
 - Eventually leads to interaction between WallRight and AlienGrid
 - Appropriate Observer is called.
 - Likewise for the WallLeft and AlienGrid

Visitor system

- All game object need to have a default behavior in base virtual class
 - { print statement – not implemented}
- Any proper collision interactions are overloaded the derived game classes.
- Visitor system helps filters specific interactions between to high level base classes.
 - Once filtered... you can call the appropriate reaction/behavior
- Need to show the pattern working

Observer System

- Register behaviors that get called when an event occurs
 - In this case – when the grid hits the wall the *observer* is called that changes direction of the march and moves the grid down at wall boundaries

Sound System

- Every time aliens march every 0.7 seconds... a different sound wave file is played
 - *fastinvader1.wav*
 - *fastinvader2.wav*
 - *fastinvader3.wav*
 - *fastinvader4.wav*

Font System

- Display at the top of the screen:
 - Score<1> Hi Score Score<2>
- Details:
 - Display what wave you are currently playing for Score<1>
 - The year the game was invented as the Hi-Score
 - Display the running number of animations for Score<2>

Approach

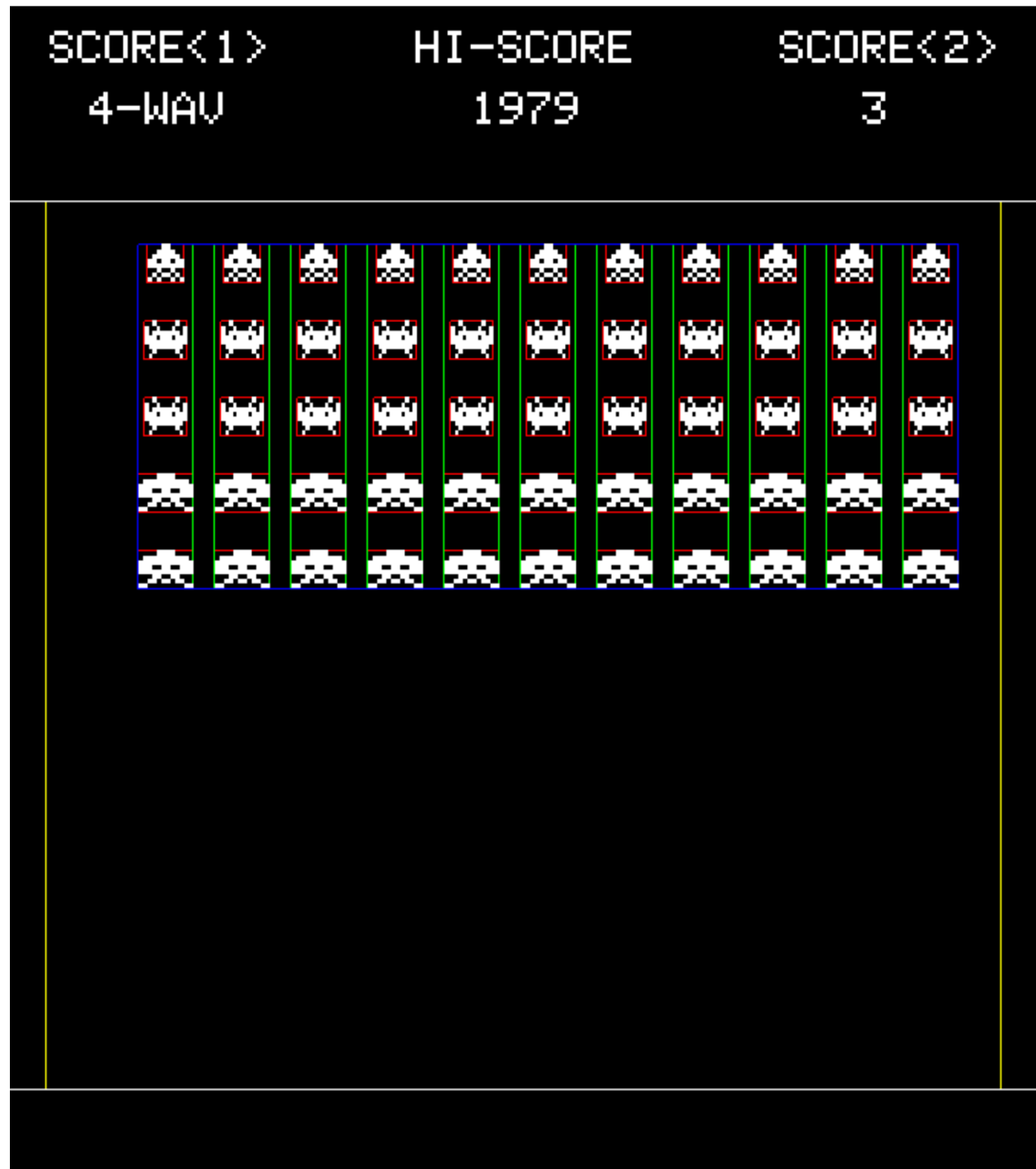
- Your choice:
 - 1st approach – start with week 7 and add MS1 material into the Sprint5
 - I think this is the easier approach
 - See detail show and tell: <https://youtu.be/MEIKrgNcvsQ>
 - 2nd approach – start with MS1 and add week 6 & 7 material into Sprint5
 - Can be confusing... a lot of merges
 - 1/2 page
 - Do not show _ALL_ the variables or the complete UML of the game
 - Only the portion that is related to the pattern
 - Need to see YOUR pattern the one actually used in your application
 - Not the Internet's

Details

- Mimic this screen:

Sprint5

— □ ×



- **Collision**

- AlienGrid and WallGroup need to be added as a single tree to the GameObjectNodeMan
- ColPair and ColPairMan dictate which trees collide and who owns the observers
 - Wall Size: 24x580 (w x h) pixels
 - Centered: y: 350 on the right and left edges of the screen

- **Fonts**

- Fonts are drawn 3x the size in the ROM... since our resolution is 3x
 - so change the font system to have a scale by 3x
- When you write a font on the screen (score, credits etc...)
 - Always use a space between the characters... for example "S C O R E < 1 >"... the actual space is small and looks good.
- Fonts are 24 pixels tall, and 15 wide.... exception "space" which is 3 pixels wide
- Top Line: (x,y)
 - 26, 740 // Score<1>
 - 265, 740 // HI-Score
 - 503, 740 // Score<2>
- 2nd line:
 - 52, 700 // wave name
 - 302, 700 // year game was invented
 - 557, 700 // animation count

- **Glyphs size**

- GlyphMan.Add(Glyph.Name.SpaceInvaders, 65, Texture.Name.SpaceInvaders, 3, 36, 5, 8); // .A
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 66, Texture.Name.SpaceInvaders, 11, 36, 5, 8); // .B
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 67, Texture.Name.SpaceInvaders, 19, 36, 5, 8); // .C
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 68, Texture.Name.SpaceInvaders, 27, 36, 5, 8); // .D
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 69, Texture.Name.SpaceInvaders, 35, 36, 5, 8); // .E
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 70, Texture.Name.SpaceInvaders, 43, 36, 5, 8); // .F
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 71, Texture.Name.SpaceInvaders, 51, 36, 5, 8); // .G
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 72, Texture.Name.SpaceInvaders, 59, 36, 5, 8); // .H
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 73, Texture.Name.SpaceInvaders, 67, 36, 5, 8); // .I
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 74, Texture.Name.SpaceInvaders, 75, 36, 5, 8); // .J
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 75, Texture.Name.SpaceInvaders, 83, 36, 5, 8); // .K
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 76, Texture.Name.SpaceInvaders, 91, 36, 5, 8); // .L
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 77, Texture.Name.SpaceInvaders, 99, 36, 5, 8); // .M
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 78, Texture.Name.SpaceInvaders, 3, 46, 5, 8); // .N
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 79, Texture.Name.SpaceInvaders, 11, 46, 5, 8); // .O
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 80, Texture.Name.SpaceInvaders, 19, 46, 5, 8); // .P
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 81, Texture.Name.SpaceInvaders, 27, 46, 5, 8); // .Q
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 82, Texture.Name.SpaceInvaders, 35, 46, 5, 8); // .R
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 83, Texture.Name.SpaceInvaders, 43, 46, 5, 8); // .S
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 84, Texture.Name.SpaceInvaders, 51, 46, 5, 8); // .T
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 85, Texture.Name.SpaceInvaders, 59, 46, 5, 8); // .U
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 86, Texture.Name.SpaceInvaders, 67, 46, 5, 8); // .V
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 87, Texture.Name.SpaceInvaders, 75, 46, 5, 8); // .W
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 88, Texture.Name.SpaceInvaders, 83, 46, 5, 8); // .X
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 89, Texture.Name.SpaceInvaders, 91, 46, 5, 8); // .Y
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 90, Texture.Name.SpaceInvaders, 99, 46, 5, 8); // .Z
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 48, Texture.Name.SpaceInvaders, 3, 56, 5, 8); // 0
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 49, Texture.Name.SpaceInvaders, 11, 56, 5, 8); // 1
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 50, Texture.Name.SpaceInvaders, 19, 56, 5, 8); // 2
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 51, Texture.Name.SpaceInvaders, 27, 56, 5, 8); // 3
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 52, Texture.Name.SpaceInvaders, 35, 56, 5, 8); // 4
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 53, Texture.Name.SpaceInvaders, 43, 56, 5, 8); // 5
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 54, Texture.Name.SpaceInvaders, 51, 56, 5, 8); // 6
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 55, Texture.Name.SpaceInvaders, 59, 56, 5, 8); // 7
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 56, Texture.Name.SpaceInvaders, 67, 56, 5, 8); // 8
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 57, Texture.Name.SpaceInvaders, 75, 56, 5, 8); // 9
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 60, Texture.Name.SpaceInvaders, 83, 56, 5, 8); // <
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 62, Texture.Name.SpaceInvaders, 91, 56, 5, 8); // >
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 32, Texture.Name.SpaceInvaders, 99, 56, 1, 8); // Space
- GlyphMan.Add(Glyph.Name.SpaceInvaders, 61, Texture.Name.SpaceInvaders, 107, 56, 5, 8); // =

```
○ GlyphMan.Add(Glyph.Name.SpaceInvaders, 42, Texture.Name.SpaceInvaders, 115, 56, 5, 8); // *  
○ GlyphMan.Add(Glyph.Name.SpaceInvaders, 63, Texture.Name.SpaceInvaders, 123, 56, 5, 8); // ?  
○ GlyphMan.Add(Glyph.Name.SpaceInvaders, 45, Texture.Name.SpaceInvaders, 131, 56, 5, 8); // -
```

- **Rendering (same as MS1)**

- Screen Size and Color
 - (width x height): **672 x 768** ← screen must be at this resolution
 - Clear Color : Black (RGB) (0,0,0) Alpha(1)
- Create a 11x5 Grid of aliens
 - Use the new **SpaceInvaders_ROM.tga**
 - **SpaceInvaders_ROM.txt** – Image dimensions are in this text file
 - Remember (in-case you forget)
 - Top row – squid
 - Middle 2 rows – crab
 - Bottom 2 rows – octopus
- Layout of Game Objects
 - GameObject (width x height)
 - Squid size: **24x 25**
 - Crab size: **28 x 25**
 - Octopus size: **36 x 25**
- Game object are arranged
 - **50 pixels** apart horizontally
 - **50 pixels** apart vertically
 - Grid of 5 rows x 11 columns
 - Bottom left Alien (Octopus)
 - start at (**86, 400**)

- **Movement**

- Aliens moving horizontally back and forth across the screen
 - Each movement is a discrete step
 - On each movement the animation changes
 - Moves step by step...
 - All aliens **animate** at **0.7 seconds** intervals
 - All aliens **move** at **0.7 seconds** intervals
 - Each alien moves +/- 4 pixels per by timer every 0.7 seconds
 - Grid moves down by **20 pixels** at wall boundaries on 0.7 seconds
 - Change direction and move down **only when collision detected by bounding boxes**
 - AlienGrid vs Walls

- **Sound playing**

- Cycle between each sound **fastinvader1.wav** → .. → **fastinvader4.wav**
- Display which sound is being played as Score<1>
- Sound plays in sync with the marching of the aliens

Required Features

Features to show off in Demo

Show it off and talk about these required features

- Collision, Visitor, Observer, Snd, Flyweight
 - Demo and talk in the video
 - Video of it running
 - Should be close to the reference demo
- Show it off
 - Be proud of what you did

Video Capture

- Have Visual Studio open and ready to go.
 - Start recording
 - Show output window running
 - Speak clearly and loud
 - Discuss the features you added in this demo
 - Duration
 - Approx. 2-3 min long YouTube video capture
 - Do not exceed 5 min
 - Use Zoom recording
 - High resolution
 - Good sound quality
 - Show on the capture... it compiling and then running
 - Place YouTube link in PDF
 - Make sure it public and runs without passwords or login
 - Watch your video
 - Can you see the code clearly
 - Can you hear your voice clearly
 - Don't lose points
 - Make sure everything is submitted properly
 - Don't record beyond 5 min.
 - Your audio volume was too quiet (test it and verify it)
 - Your YouTube link needs to work "As-IS" with no login
 - Don't make it private... make it unlisted
 - You check-in too many files or temporary files
- Make sure you run CleanMe.bat before perforce submission

Validation

Simple checklist to make sure that everything is submitted correctly

- Is the project compiling and running without any errors or warnings?
- Does the project runs **ALL** without crashing?
- Is the submission report filled in and submitted to performe?
- **Follow the verification process** for performe
 - Is all the code there and compiles “as-is”?
 - No extra files
- **Submit the YouTube link to PDF**

Hints

Most assignments will have hints in a section like this.

- Watch the lecture... look at the code supplied in class
- Study the projects in class lecture

Troubleshooting

- Print to the output window to help you debug
 - It really helps
- Ask for clarification on piazza